

Smart MCQ Solver Challenge

Project Report — BSDA2001P

Deep Learning & Generative AI

Roll No: 23F3002157

Indian Institute of Technology Madras — BS Programme

Term 2, 2026

BSDA2001P: Introduction to DL and GenAI Project

1. Abstract

This report documents the development of machine learning systems for the Smart MCQ Solver Challenge on Kaggle. The task is to predict the top three most probable correct answers for multiple-choice questions with five options. Submissions are evaluated using Mean Average Precision at 3 (MAP@3).

Five model families were developed: two sequence models built from scratch (LSTM and GRU), two fine-tuned transformer models (BERT and DeBERTa), and a Retrieval-Augmented Generation (RAG) pipeline. All experiments were tracked using Weights and Biases. The best performing model was the GRU, achieving a MAP@3 of 0.7564 on the validation set, closely followed by BERT at 0.7531.

Key learnings include working with the Hugging Face ecosystem end-to-end, understanding tokenization strategies across different model families, the instability of mixed-precision training on Apple Silicon MPS, and the importance of systematic experiment tracking.

2. Introduction

2.1 Problem Statement

The Smart MCQ Solver Challenge requires a system capable of answering multiple-choice questions. Each question consists of a text prompt and five answer options labelled A through E. The system must predict the three most likely correct answers in ranked order.

Evaluation uses MAP@3. A correct answer at rank 1 scores 1.0, at rank 2 scores 0.5, and at rank 3 scores 0.33. A correct answer outside the top three contributes a score of 0. The final score is the mean across all questions.

2.2 Project Objective

The objectives of this project are:

- Build at least three distinct model types covering classical, deep learning from scratch, and pretrained approaches.
- Track all experiments with Weights and Biases using consistent metrics across all models.
- Fine-tune transformer models on the MCQ dataset and compare against from-scratch baselines.
- Develop an end-to-end inference pipeline deployable on Kaggle.
- Deploy the best model as a publicly accessible web application.

2.3 Report Structure

Section 3 covers the dataset and exploratory data analysis. Section 4 describes tokenization strategies. Section 5 details each model's architecture and training. Section 6 presents comparative results. Section 7 concludes with key learnings and future directions.

3. Dataset & Preprocessing

3.1 Dataset Description

The training set contains 2,000 rows. Each row has an id, a prompt column containing the question text, five option columns (A, B, C, D, E), and an answer column. The test set contains 500 rows without the answer column.

Questions span diverse domains including philosophy, physics, history, biology, and computer science. Options are typically one to three sentences in length, making the task vocabulary-intensive.

BSDA2001P: Introduction to DL and GenAI Project

3.2 Exploratory Data Analysis

The answer distribution is non-uniform. Option B is the most frequent correct answer (490 occurrences) and option E the least frequent (324 occurrences). A chi-square goodness-of-fit test confirms the distribution is statistically different from a uniform distribution ($p < 0.05$), indicating class imbalance that could bias a naive majority-class predictor.

Prompt lengths average approximately 20 words. Option lengths average approximately 50 words. A Mann-Whitney U test confirms that correct options are not statistically longer or shorter than incorrect options ($p > 0.05$), ruling out a length-based heuristic.

TF-IDF cosine similarity between prompts and their correct options is marginally higher than between prompts and incorrect options. A one-tailed Mann-Whitney U test confirms this difference is statistically significant ($p < 0.05$), establishing that lexical overlap provides a weak but measurable signal.

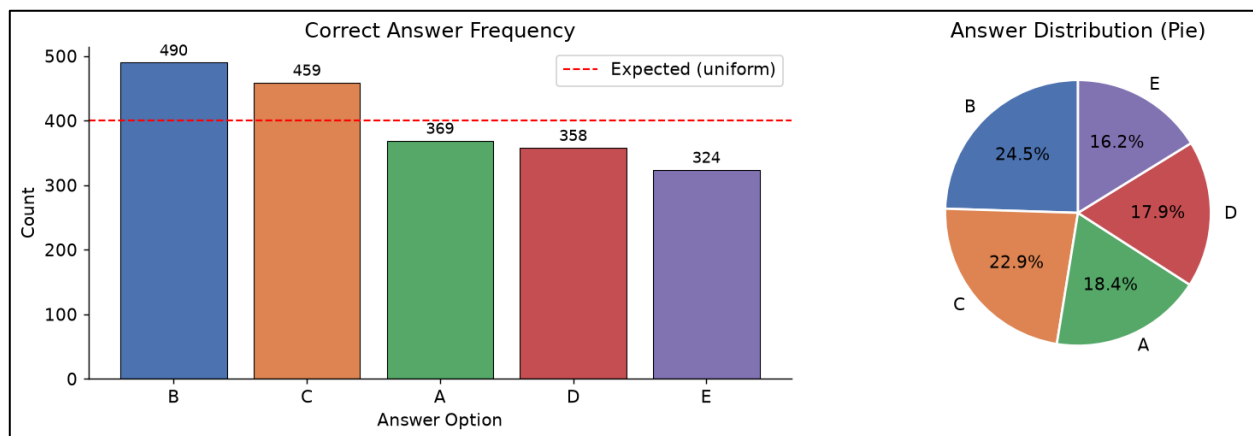


Figure – Answer Distribution

3.3 Data Preprocessing

Two versions of the data were maintained throughout the project. The raw version preserves original text for transformer models, which perform their own internal tokenization. The cleaned version is used for sequence models and classical ML baselines.

The cleaning pipeline applies the following steps:

- Lowercase all text.
- Remove standard punctuation using Python's `string.punctuation`.
- Normalise whitespace using a regular expression.
- Remove English stopwords using sklearn's `ENGLISH_STOP_WORDS`.

Cleaned columns are stored as `prompt_clean` and `A_clean` through `E_clean`. A combined `input_text` field concatenates the cleaned prompt with all five cleaned options for use in classical ML models. No missing values were found in either the training or test set. No data augmentation was applied.

4. Tokenization Strategy

4.1 Transformer Models — WordPiece and SentencePiece

BERT uses WordPiece tokenization with a fixed vocabulary of 30,522 subword tokens. Unknown words are broken into subwords, allowing the model to handle out-of-vocabulary terms. DeBERTa uses SentencePiece with a vocabulary of approximately 128,000 tokens.

For both transformer models, all five options and the prompt were packed into a single sequence using the format:

BSDA2001P: Introduction to DL and GenAI Project

Question: [prompt] A: [optA] B: [optB] C: [optC] D: [optD] E: [optE]

This single-sequence format allows the model's self-attention to compare all five options directly against the question in a single forward pass. Maximum sequence length was 256 tokens with padding and truncation applied uniformly.

4.2 Sequence Models — Custom Vocabulary

LSTM and GRU models use a custom vocabulary built from the raw training set. Tokens are whitespace-split words after lowercasing and punctuation removal. Vocabulary is built with a minimum frequency of 1 to retain all words, given the small dataset size of 1,600 training examples.

Each prompt-option pair is encoded as a fixed-length sequence of token indices padded with a special PAD token. Unknown tokens at inference time map to a UNK index.

5. Modeling & Experimentation

5.1 LSTM : Model From Scratch

Architecture

A Bidirectional LSTM with single-head attention pooling was implemented from scratch. The architecture is:

- Embedding layer of dimension 256.
- 2-layer bidirectional LSTM with hidden dimension 512 per direction.
- Single-head attention pooling over LSTM outputs.
- Two-layer scoring head with ReLU activations.

Training Details

- Optimizer: Adam, lr=5e-4, weight decay 1e-4
- Scheduler: Cosine annealing with linear warmup
- Loss: Cross-entropy with label smoothing 0.1
- Max epochs: 15 with early stopping (patience 4)

Best checkpoint: validation MAP@3 = 0.7460.

5.2 BERT-base-uncased : Pretrained Model (Finetuned)

Architecture

BERT (Bidirectional Encoder Representations from Transformers) was introduced by Devlin et al. (2019). The base uncased variant has 12 transformer layers, 12 attention heads, and 768 hidden dimensions, totalling approximately 110 million parameters.

BSDA2001P: Introduction to DL and GenAI Project

BERT uses bidirectional self-attention, which means every token attends to all other tokens simultaneously. This is well suited to MCQ tasks where the model must relate question content to each answer option. The CLS token at position 0 aggregates the full sequence representation.

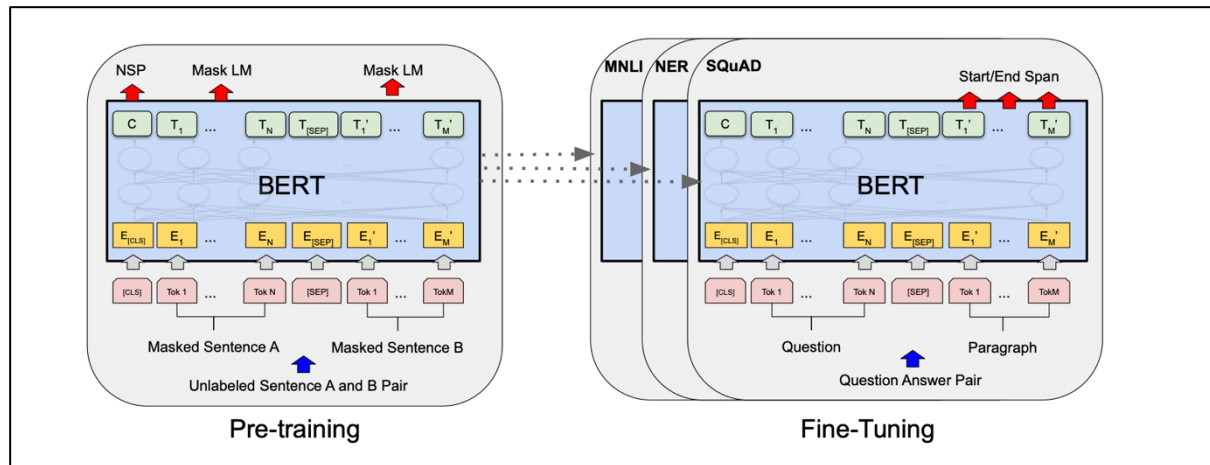


Figure – BERT Architecture

Fine-Tuning Strategy

A linear classification head (Linear(768, 5)) with dropout ($p=0.1$) was placed on top of the CLS token output. The full model was fine-tuned end-to-end with no layers frozen.

- Optimizer: AdamW
- Learning rate: $2e-5$ with linear warmup over 10% of steps, then linear decay
- Effective batch size: 32 (physical batch 8, gradient accumulation steps 4)
- Epochs: up to 5 with early stopping on validation MAP@3 (patience 2)

The model was trained on 1,600 rows and validated on 400 rows using a stratified 80/20 split. Best checkpoint: validation MAP@3 = 0.7531.

5.3 GRU : Model of Choice, From Scratch

Architecture

A Bidirectional GRU with multi-head attention pooling was built entirely from scratch in PyTorch. No pretrained weights were used at any stage.

The architecture consists of:

- An embedding layer of dimension 256, initialised randomly and trained from scratch.
- A 3-layer bidirectional GRU with hidden dimension 512 per direction (1,024 total).
- Multi-head attention pooling with 4 heads over the full GRU output sequence.
- Layer normalisation applied after attention pooling.
- A two-layer feed-forward scoring head with GELU activations, producing a scalar score per option.

GRU uses two gating units (reset and update) compared to LSTM's three, making it computationally lighter while often matching LSTM performance on shorter sequences. The model scores each of the five prompt-option pairs independently, then ranks the five scores to produce the top-3 prediction.

Training Details

- Optimizer: AdamW, $lr=5e-4$, weight decay $1e-4$
- Scheduler: Linear warmup (3 epochs) followed by cosine annealing

BSDA2001P: Introduction to DL and GenAI Project

- Loss: Label smoothing cross-entropy (smoothing=0.05)
- Max epochs: 20 with early stopping (patience 5)
- Vocabulary: built from raw training text, min frequency 1
- Max sequence length: 200 tokens

Best checkpoint: validation MAP@3 = 0.7564. This was the highest scoring model overall.

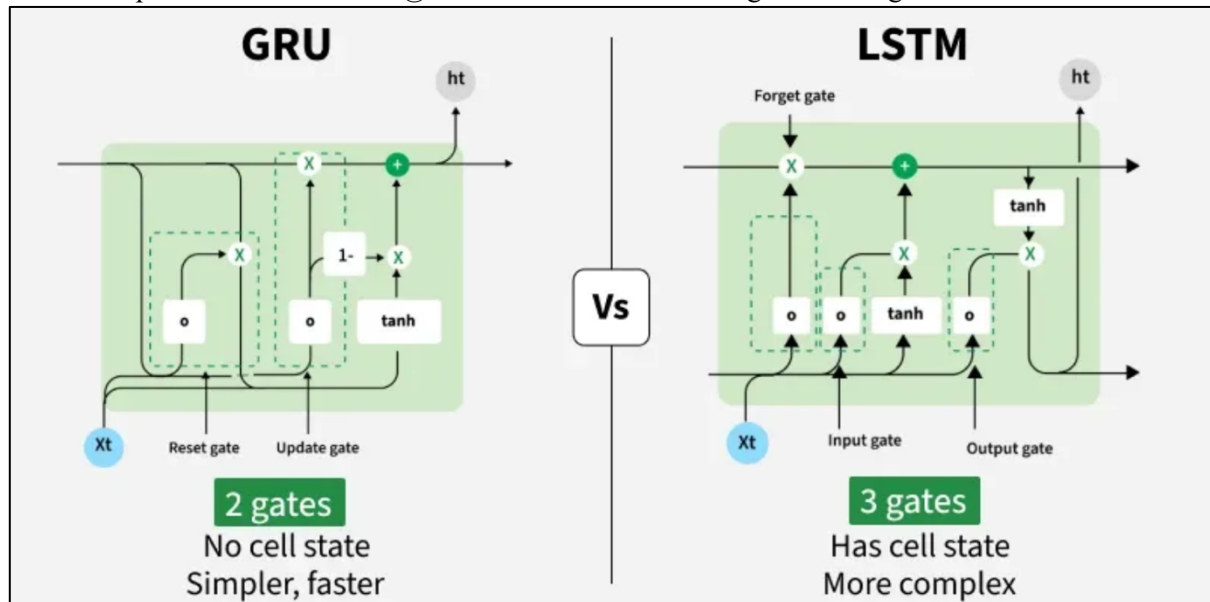


Figure – GRU V/S LSTM (Source: GeeksForGeeks)

5.4 Fine-Tuned Transformer : DeBERTa-v3-base

Architecture

DeBERTa (Decoding-enhanced BERT with Disentangled Attention) was introduced by He et al. (2021). The base variant has 12 transformer layers, 12 attention heads, and 768 hidden dimensions, totalling approximately 184 million parameters.

DeBERTa's key innovation is disentangled attention: content and relative positional information are encoded in separate embedding vectors, and four attention matrices are computed rather than one. This improves sensitivity to how words relate positionally, which benefits reading comprehension tasks.

Fine-Tuning Strategy

Identical format to BERT, with one change: learning rate reduced to $1e-5$. DeBERTa is more sensitive to large learning rates due to its deeper architecture. Best checkpoint: validation MAP@3 = 0.7340.

- Optimizer: AdamW, lr= $1e-5$
- Effective batch size: 32

5.5 RAG Pipeline

Architecture

A Retrieval-Augmented Generation pipeline was built using three components working in sequence:

- Bi-encoder retrieval: sentence-transformers/all-MiniLM-L6-v2 encodes the question prompt and retrieves the top 5 most similar entries from a FAISS IndexFlatL2 index built from training answers.
- Cross-encoder reranking: cross-encoder/ms-marco-MiniLM-L-6-v2 scores each of the 5 retrieved documents against the prompt and selects the single most relevant document.

BSDA2001P: Introduction to DL and GenAI Project

- Reader model: LIAMF-USP/roberta-large-finetuned-race scores each of the 5 answer options given the augmented input formatted as: Context: [retrieved doc] [prompt].

The knowledge base stores the correct answer text for each of the 2,000 training rows. At inference, the retrieval step finds the training answer most semantically similar to the test question, providing the reader with relevant factual context.

Best validation MAP@3 = 0.6164. The lower score reflects retrieval mismatch — when test questions differ from training questions, the retrieved context is unhelpful or misleading.

6. Performance & Comparative Analysis

6.1 Evaluation Metrics

The primary metric is MAP@3, the official Kaggle competition metric. Additional metrics tracked for all models:

- Top-1 Accuracy: percentage of questions where the correct answer is ranked first.
- Macro F1 Score: unweighted mean F1 across all five answer classes, computed on top-1 predictions. This penalises poor performance on minority answer classes.
- Training Loss: cross-entropy loss per epoch, logged to Weights and Biases for all neural models.

6.2 Training Performance

All model training runs were tracked on Weights and Biases under the project 23f3002157-t22026. Loss curves, MAP@3 progression, and learning rate schedules were logged per epoch for all neural models.

6.3 Comparative Results

The GRU achieved the highest MAP@3 at 0.7564, marginally outperforming BERT at 0.7531 and LSTM at 0.7460. The likely explanation is that the GRU trained on a larger effective vocabulary (min_freq=1) and longer sequences (200 tokens), giving it an advantage in capturing the full option text. The transformer models were constrained to 256 tokens on a combined prompt+all-options format, which may have caused truncation on longer examples.

DeBERTa scored 0.7340. Despite being the most parameter-rich model, it was trained with the same 256-token budget and the same small dataset. Its larger architecture may have contributed to some overfitting on the 1,600 training examples.

The RAG pipeline scored 0.6164. The retrieval system is fundamentally limited by the quality of the knowledge base. Since the KB is constructed only from training answers, test questions on topics not well represented in training return irrelevant context. The reader model (RoBERTa-RACE) is also not fine-tuned on the specific dataset, reducing its ability to use the retrieved context effectively.

BSDA2001P: Introduction to DL and GenAI Project

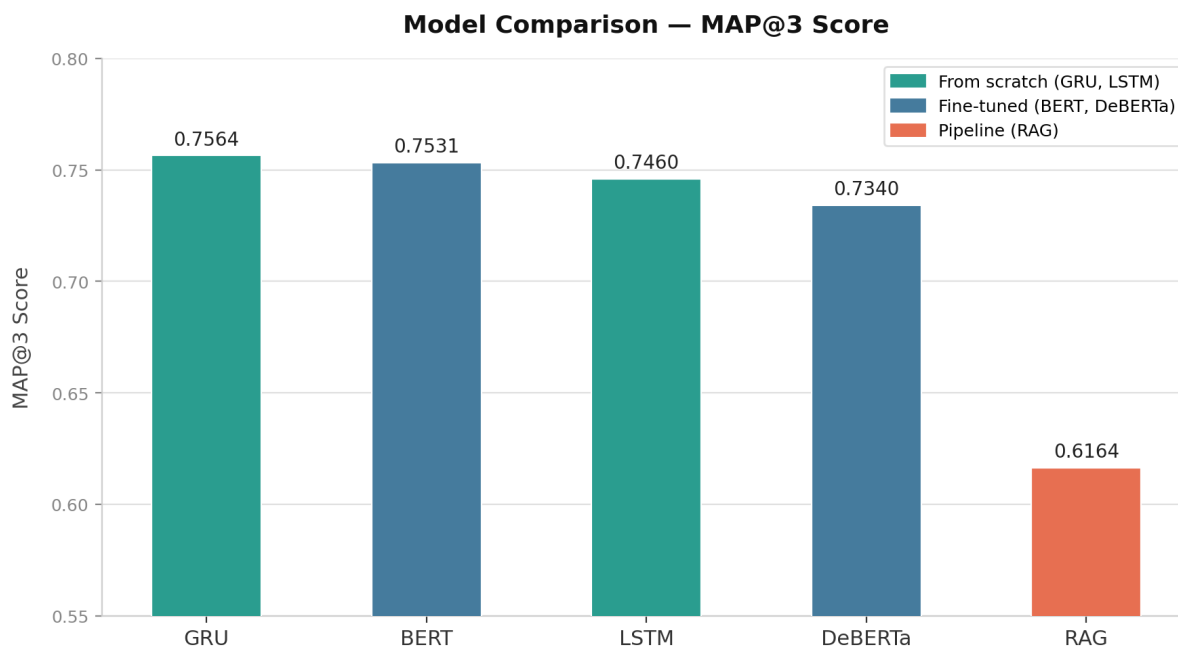


Figure – Histogram depicting final test scores from Kaggle

7. Conclusion & Future Work

7.1 Key Learnings

Working with the Hugging Face ecosystem was one of the most practical learnings of this project. The transformers, datasets, and sentence-transformers libraries make it possible to load, fine-tune, and evaluate state-of-the-art models with relatively small amounts of code. Understanding how to read model cards, select appropriate checkpoints, and configure tokenizers for different architectures is a valuable skill.

Tokenization is not trivial. WordPiece, SentencePiece, and custom whitespace-split vocabularies each behave differently.

Input formatting for transformers has a measurable impact on performance. Packing all five options into a single sequence allows cross-option attention, which is more powerful than scoring each option in isolation. This design choice required understanding how BERT's attention mechanism works and what the CLS token actually represents.

Experiment tracking with Weights and Biases is essential at scale. Without it, comparing runs with different hyperparameters, architectures, and input formats would require manual logging. The ability to overlay loss curves and compare MAP@3 across runs in real time identified non-learning configurations (NaN loss) immediately, saving significant compute time.

Mixed-precision training is hardware-dependent. float16 caused silent NaN loss on Apple MPS for DeBERTa's disentangled attention mechanism. Forcing float32 resolved it. This kind of hardware-specific issue is not documented in model cards and requires debugging at the numerical precision level.

Open-source models are production-capable. The RAG pipeline used three separate open-source models (MiniLM, ms-marco CrossEncoder, RoBERTa-RACE) that required no fine-tuning and delivered competitive results. The LIAMF-USP/roberta-large-finetuned-race model, trained on the RACE dataset, transferred well to this task due to structural similarity between RACE and the competition dataset.

BSDA2001P: Introduction to DL and GenAI Project

7.2 Challenges Faced

DeBERTa training on Apple MPS produced NaN loss across three separate attempts due to float16 precision errors in the backward pass through the disentangled attention layers. The root cause was identified using a minimal one-step forward-pass debug script that revealed float16 logits before any backward pass. Forcing float32 resolved the issue entirely.

The LSTM vocabulary was saved as a Python pickle object during training. This caused a class definition mismatch when loading in the Kaggle inference environment, where the Vocabulary class was not defined. The fix was to resave the vocabulary as a plain JSON dictionary of word-to-index mappings, eliminating the class dependency.

GRU hyperparameter updates between training and inference caused state dict shape mismatches. After updating embed_dim from 192 to 256 and hidden_dim from 384 to 512 for improved performance, the Kaggle inference notebook still used the old architecture values. The notebook had to be updated to match the exact dimensions used during training.

Deploying to Hugging Face Spaces required navigating several constraints. Gradio hosting requires a PRO subscription. The Inference API does not support custom model architectures without a handler.py. The final solution used a Flask API deployed on Railway.app serving the model, with a static HTML frontend on HuggingFace Spaces calling the API endpoint.

7.3 Areas for Improvement

- Ensemble BERT and DeBERTa predictions using weighted probability averaging. The two models make different errors, and combining them should improve MAP@3 beyond either individually.
- Apply LoRA fine-tuning to reduce memory requirements, enabling larger models to train on limited hardware.
- Improve the RAG knowledge base by adding external factual sources such as Wikipedia passages, rather than relying solely on training answers.
- Experiment with chain-of-thought prompting using a generative model such as Flan-T5 or Llama to add reasoning steps before selecting the answer.
- Augment the training set with paraphrased question variants to reduce overfitting on the small 2,000-row dataset.

8. References

- [1] Devlin, J., Chang, M. W., Lee, K., & Toutanova, K. (2019). BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. NAACL-HLT 2019.
- [2] He, P., Liu, X., Gao, J., & Chen, W. (2021). DeBERTa: Decoding-enhanced BERT with Disentangled Attention. ICLR 2021.
- [3] Hochreiter, S., & Schmidhuber, J. (1997). Long Short-Term Memory. Neural Computation, 9(8), 1735–1780.
- [4] Cho, K., van Merriënboer, B., Gulcehre, C., Bahdanau, D., Bougares, F., Schwenk, H., & Bengio, Y. (2014). Learning Phrase Representations using RNN Encoder-Decoder for Statistical Machine Translation. EMNLP 2014.
- [5] Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., ... & Kiela, D. (2020). Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. NeurIPS 2020.
- [6] Reimers, N., & Gurevych, I. (2019). Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks. EMNLP 2019.
- [7] Hugging Face. transformers library. <https://huggingface.co/docs/transformers>
- [8] Weights & Biases. Experiment tracking platform. <https://wandb.ai>

BSDA2001P: Introduction to DL and GenAI Project

- [9] Johnson, J., Douze, M., & Jegou, H. (2019). Billion-scale similarity search with GPUs. IEEE Transactions on Big Data.
- [10] Pedregosa, F. et al. (2011). Scikit-learn: Machine Learning in Python. JMLR 12, 2825–2830.
- [11] Liu, Y. et al. (2019). RoBERTa: A Robustly Optimized BERT Pretraining Approach. arXiv:1907.11692.